

A Brief Analysis of Consensus Protocol: From Logical Clock to Raft

Alibaba Clouder

April 11, 2019

 24,627 0

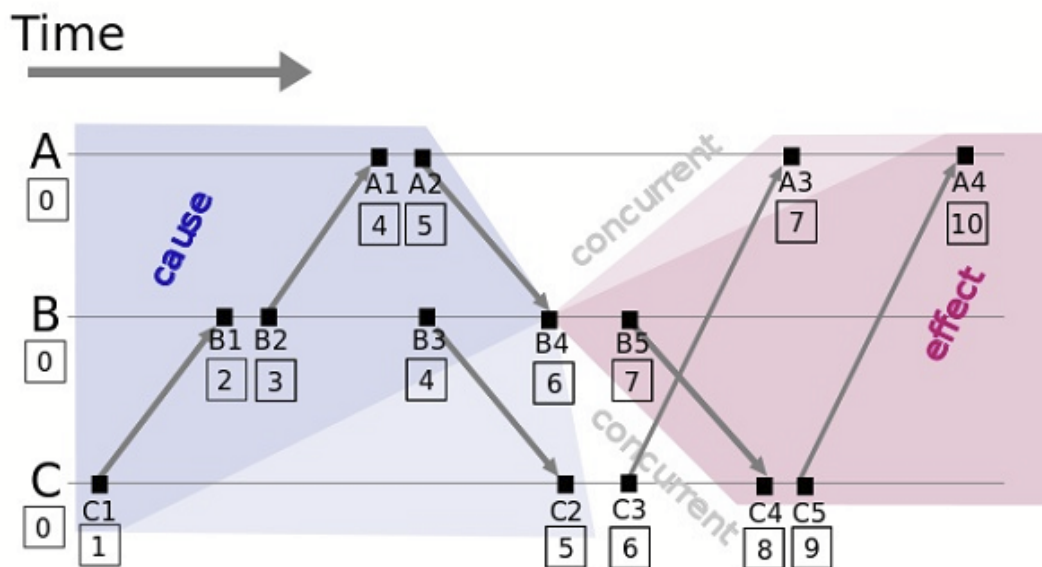
This article explores the concepts and benefits of generic programming and serialization through a simple example.

During this year's Spring Festival, I read some papers about consensus protocols. Before reading these papers, I have always been wondering what the actual differences are between ZooKeeper's ZAB protocol and the Raft protocol, which both have leader and two-phase commit, and how the Paxos protocol can be applied in actual engineering. These papers provided me with answers to these questions. Now I try to explain these protocols in my own words to help you understand these algorithms. At the same time, I also mention some questions in this article and hope we can discuss them together. My knowledge of these protocols may be limited, so please correct me where I am wrong.

Logical Clock

A logical clock is not actually a consensus protocol. It is an idea put forward by Lamport in 1987 to solve possible problems caused by clock inconsistency between different machines in a distributed system. In a standalone system, machine time is used to identify events so that we can clearly identify the order of occurrence for two different events. However, in a distributed system, because time deviation may vary from machine to machine, event order cannot be decided through physical clocks. In fact, in a distributed system, we only pay attention to the order of two associated

events. Consider two transactions: One is the modification of row A and the other is the modification of row B. We actually don't care which of the two transactions occurs first. The so-called logical clock is what is used to define the order of occurrence for two associated events, that is, "happens before". A logical clock cannot determine the order of occurrence for unassociated events. So, this "happens before" relation is actually a partial ordering relation.



Diagrams and examples in this article are from [this blog](#).

In this diagram, the arrow indicates inter-process communication; A, B, and C indicate the three processes in a distributed system.

The algorithm of the logical clock is very simple: Each event corresponds to a Lamport timestamp (the initial value is 0).

If an event occurs within a node, the timestamp value adds 1.

If an event is a sending event, the timestamp adds 1 and the timestamp is added to that message.

If an event is a reception event, the timestamp is Max (local timestamp within a message) plus 1.

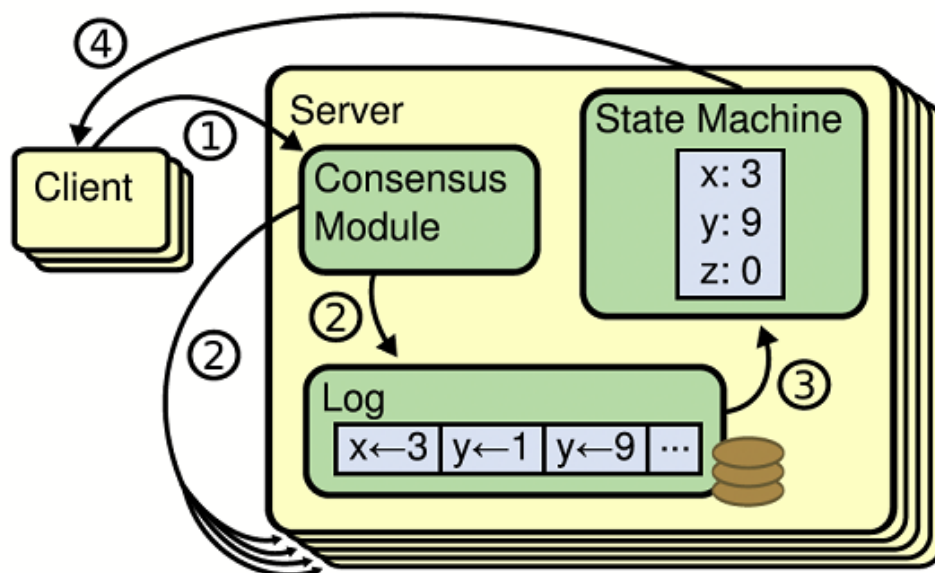
For all associated sending and reception events, this can ensure that the timestamp of a sending event is smaller than that of a reception event. If

two events (for example, A3 and B5) are not associated, they have the same logical time. We can define the occurrence order of the two events as we like because the two events are not associated. For example, if process A happens before process B and process C when the Lamport timestamp is the same, we can reach the conclusion that "A3 happens before B5". However, in the physical world, B5 happens before A3. This does not matter, though.

It seems that currently logical clocks are not widely applied. DynamoDB uses the vector clock to solve the chronological order of multiple versions. Please notify me of any other specific applications that I'm not aware of. Spanner by Google also uses a physical clock to solve the clock issue. However, we can see the prototype of consensus protocols from the logical clock algorithm by Lamport.

Replicated State Machine

When it comes to consensus protocols, we usually talk about the state machine replication. Usually state machine replication and consensus protocol algorithms are used together to implement high availability and fault tolerance in distributed systems. Many distributed systems use state machine replication to synchronize data between copies, such as HDFS, Chubby, and Zookeeper.



State machine replication maintains a persisted log in each instance copy in

a distributed system and uses a certain consensus protocol algorithm to ensure that the log is completely consistent within each instance. In this way, the state machine within instances plays back each command in the log by log sequence so that the same data is read from each copy when a client reads data. The core of the state machine replication is the Consensus module shown in the figure, which is included in the consensus protocol algorithms that we are going to discuss today.

Paxos

Paxos is a consensus protocol algorithm developed by Lamport in the 1990s. It was widely found hard to understand. Therefore, Lamport published a new paper "Paxos Made Simple" in 2001, where he said that Paxos is the simplest consensus algorithm in the world and very easy to understand. However, it is still generally considered hard to understand in this industry. After reading Lamport's papers, I think that the Paxos protocol itself is not hard to understand apart from the complex process of argument for correctness. However, the Paxos protocol is too theoretical and far from being applied in specific engineering practices. I was also very confused when I first learned about the Paxos protocol. I read the papers many times and found that this protocol is just for single-event consensus and that the agreed value cannot be modified. How can we use Paxos to implement state machine replication? In addition, only Proposer and Follower know the agreed values based on the Paxos protocol. How can we actually use this protocol? However, the Paxos protocol is a lot easier to understand if you only consider this protocol theoretically and do not consider problems that may occur in actual engineering. In Lamport's papers, the application of state machines is just a general idea and no specific implementation logic is included. It is impossible to directly use Paxos for state machine replication. Instead, we need to add many things to Paxos. That is why Paxos has so many variants.

Basic Paxos

Basic Paxos is the Paxos algorithm first put forward by Lamport. In fact, it is simple and can be explained in just a few words. Next, I will describe Paxos in my own words and give an example. To understand Paxos, simply remember one thing: Paxos can only enable consensus for one value and the proposal cannot be changed once decided. That is to say, the entire Paxos Group only accepts one proposal (or several proposals with different



values). As to how to accept multiple values to implement state machine replication, see Multi Paxos in the next section.

The Paxos protocol does not have the Leader concept. Apart from Learner (only learning the result of Propose; not to be discussed here), only Proposer and Acceptor are available in Paxos. Paxos allows multiple Proposers to propose a number at the same time. A proposer proposes one value and tries to convince all Acceptors to agree on the value. In the Prepare phase, a Proposer sends a ProposalID n to each Acceptor (note that the Proposer will not pass the value to an Acceptor in this phase). If an Acceptor finds that it has never received a value greater than n from any of the Proposers, it will reply to the Proposer and promise not to receive the Prepare messages of proposals with ProposalID smaller than or equal to n . If an Acceptor has already promised a proposed number greater than n , it will not reply to the Proposer. If the Acceptor has accepted a proposal less than n (in phase 2), it will return this proposed value to the Proposer. Otherwise, a null value is returned. When the Proposer receives replies from more than half of the Acceptors, it is ready to start the Accept phase. However, in this phase a value that can be proposed by a Proposer is limited. The Proposer can propose a new value if and only if the replies received do not include values of previous proposals. Otherwise, the Proposer can only use the greatest proposal value among the replies as the proposal value. The Proposer uses this value and ProposalID n to launch the Accept request against each Acceptor. That is to say, even if the Proposer has already received a promise from an Acceptor, the Accept may fail because the Acceptor may have given a promise to a Proposer with a higher proposalID before the Accept is initiated. In other words, the second phase may still fail due to the presence of multiple Proposers, even if the first phase succeeds.

I'll provide an example, which is from [this blog](#).

Consider three servers: Server1, Server2, and Server3. They all want to use the Paxos protocol to make all members agree that they are leaders. These servers are the Proposer role and the values that they propose are their names. They need consent from the three members: Acceptor 1-3. Server2 initiates proposal 1 (with 1 as the ProposalID), Server1 initiates proposal 2 and Server3 initiates proposal 3.

First, it is the Prepare phase:

Assume that the message sent from Server1 arrives at Acceptor 1 and

Acceptor 2 first, both of which have not received a request. Therefore, they receive this request and return [2, null] to Server1. At the same time, they promise not to receive requests with an ID smaller than 2;

Then the message sent from Server2 arrives at Acceptor 2 and Acceptor3, and Acceptor 3 has not received a request. Therefore, Acceptor 3 returns [1, null] to Proposer 2 and promises not to receive messages with an ID smaller than 1. Because Acceptor 2 has already received the request from Server1 and promised not to receive requests with an ID smaller than 2, Acceptor 2 will refuse the request from Server2.

Finally, the message from Server3 arrives at Acceptor 2 and Acceptor 3, both of which have already received a proposal. However, because this message has an ID that is greater than 2 (the ID of the message that has been received by Acceptor 2) and 1 (the ID of the message that has been received by Acceptor 3), both Acceptor 2 and Acceptor 3 receive this proposal and return [3, null] to Server3.

At this point, because Server2 does not receive more than half of the replies, it obtains a new message with 4 as its ID and sends this message to Acceptor 2 and Acceptor 3. Since 4 is greater than 3 (the maximum ID of the proposals that Acceptor 2 and Acceptor 3 have received), this proposal is received and [4, null] is returned to Server2.

Next, it is the Accept phase:

Because Server3 has received more than half of the replies (2 replies) and the returned value is null, Server3 submits the proposal [3, server3].

Because Server1 has also received more than half of the replies in the Prepare phase and the returned value is null, Server1 submits the proposal [2, server1].

Because Server2 has also received more than half of the replies and the returned value is null, Server2 submits the proposal [4, server2].

When Acceptor 1 and Acceptor 2 receive the proposal [2, server1] from Server1, Acceptor 1 accepts this request and Acceptor 2 refuses this request because it has promised not to accept a proposal with an ID smaller than 4.

Acceptor 2 and Acceptor 3 accept the proposal [4, server2] from Server2 when they receive that proposal.



Acceptor 2 and Acceptor 3 will refuse the proposal [3, server3] from Server3 because they have promised not to accept a proposal with an ID smaller than 4.

At this point, more than half of the Acceptors (Acceptor 2 and Acceptor 3) have accepted the proposal [4, server2]. The Learner perceives that the proposal is passed and starts to learn the proposal. Therefore Server2 becomes the final leader.

Multi-Paxos

As mentioned previously, Paxos is in the theoretical phase and cannot be directly used for state machine replication. The reasons are as follows:

- Paxos can only determine one value and cannot be used for continuous log replication.
- The presence of multiple Proposers may lead to livelock. In the previous example, Server2 submits a proposal twice before a proposal is finally accepted. In some extreme scenarios, more submittals of proposals may be required.
- The final result of a proposal is only known to partial Acceptors. This cannot guarantee that each instance for state machine replication has a completely consistent log.

In fact, Multi-Paxos is actually used to solve the three aforementioned problems, so that the Paxos protocol can be used in state machines. It is very simple to solve the first problem. Each index value of a log entry uses an independent Paxos instance. It is also easy to solve the second problem: Include only one Proposer in a Paxos group, select a Leader by using the Paxos protocol at the time of reading (as shown in the previous example) and let this Leader perform reading to avoid the livelock problem.

Additionally, a single Leader allows us to omit most work in the Prepare phase. After a Leader is selected, simply start the Prepare once. If none of the Acceptors has received Prepare requests from other Leaders, accept requests directly each time a request is written, unless an Acceptor refuses the request (refusal indicates that a new Leader is performing writing). To solve the third problem, Multi-Paxos adds a firstUnchosenIndex to each server and lets a Leader synchronize selected values to each Acceptor. After these problems are solved, Paxos can be used in actual engineering.

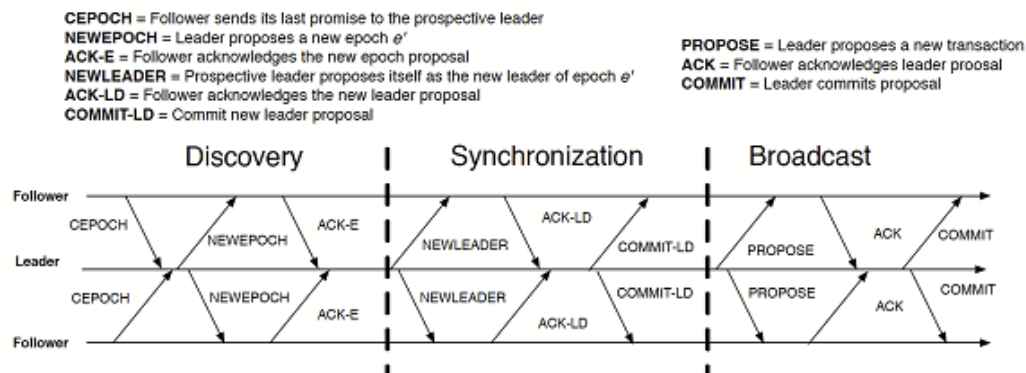
Paxos has many additions and variants so far. In fact, ZAB and Raft, which I

will discuss later, can be seen as modifications and variants of Paxos. A widespread remark says that "Only one consensus algorithm exists in the world, that is, Paxos."

ZAB

ZAB (ZooKeeper Atomic BoardCast) is a consensus protocol used in ZooKeeper. ZAB is a dedicated protocol for Zookeeper. It is strongly bound to Zookeeper and has not been extracted into an independent database. Therefore, ZAB is not widely used and only limited to Zookeeper. However, the papers on ZAB protocol thoroughly prove that ZAB has the ability to meet the strong consistency requirement.

ZAB was born in 2007 along with Zookeeper. The Raft protocol had not been developed that time. According to the papers on ZAB, Zookeeper did not directly use Paxos but developed its own protocol because Paxos was considered incapable of meeting the requirements of Zookeeper. For example, Paxos allowing multiple Proposers may cause multiple commands submitted from a client to fail to be executed by FIFO sequence. Additionally, in the recovery process, the data of some followers may be incomplete. These arguments are based on the original Paxos protocol. In fact, these problems have been solved in some variants of Paxos. For historical reasons, the original Paxos protocol failed to solve the aforementioned problems. Therefore Zookeeper developers decided to develop a new consensus protocol—ZAB.



ZAB is very similar to the subsequent Raft protocol. ZAB handles selecting a leader as well as recovery. Writing is also performed by using a two-phase commit. First, a round of votes are initiated from a leader. After the

acceptance of more than half of the votes, a commit is launched. The epoch number of each leader in ZAB is actually equivalent to the term in Raft that I will talk about later. However, in ZAB, the epoch number and the transition number constitute a zxid, which is stored in each entry.

ZAB enables log replication by using a two-phase commit. The first phase is vote. This phase is successfully complete when more than half of the consent votes are obtained. In this phase, data is not really transferred to followers. This is to ensure that more than half of the machines are working correctly or within the same network partition. The second is the commit phase. In this phase, data is transferred to each follower and then each follower (as well as the leader) appends data to the log. At this point, the write operation is completed. It does not matter if voting in the first phase is successfully done but a follower fails in the second phase. Restarting the leader can ensure that the data of followers is consistent with that of the leader. If the leader fails in the commit phase and this write operation has been committed on at least one follower, this follower will definitely be selected as a leader because its zxid is the greatest. After being selected as the leader, this follower lets all followers commit this message. If no followers have committed this message when the leader fails, this write operation is not completed.

Because logs only need to be appended at commit, ZAB logs only require the append-only capability.

Additionally, ZAB supports stale reads from replicas. To implement strongly consistent reading, we can use sync read. Here is how it works: First, a virtual write operation is launched (nothing is written). After this operation is completed, this sync operation is also committed locally. Then reading is performed on the local replicas to ensure that all the data before this sync time point is correctly read. However, reads and writes under the Raft protocol are performed through primary nodes.

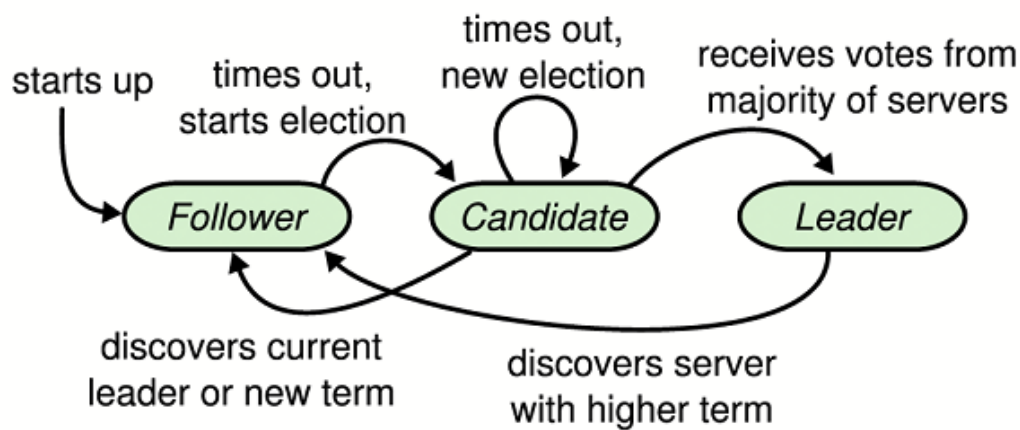
Raft

Raft is a new consensus protocol developed by developers at Stanford University in 2014. The developers developed this new consensus protocol because they considered Paxos difficult to understand. In addition, Paxos is only a theory and far from being applied in actual engineering. The developers of Paxos listed some disadvantages of Paxos:



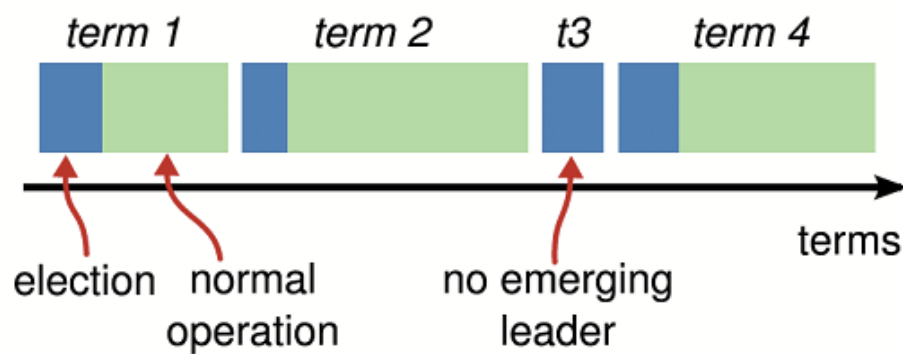
1. The Paxos protocol does not require a leader. Each Proposer can create a proposal. Leader selection and consensus agreement are separated at the very beginning of designing Raft, while leader selection and proposal are mixed together in Paxos, making Paxos hard to understand.
2. The original Paxos protocol is only to reach consensus on one single event. Once a value is determined, it cannot be modified. However, in realistic scenarios (including database consistency), it is required to continuously reach consensus on the value of a log entry. Therefore, the Paxos protocol itself cannot meet the requirement: We need to make some improvements and supplements to the Paxos protocol to apply Paxos in engineering in a real sense. Making supplements to the Paxos protocol is very complex. Although the Paxos protocol has been proven by Lamport, the Paxos-based and improved algorithms like Multi-Paxos are unproven.
3. Another disadvantage is that Paxos only provides a rough description. This requires that subsequent improvements on Paxos and projects that use Paxos like Google Chubby have to implement a set of projects to solve specific problems in Paxos. The implementation details of projects like Chubby are not made public. That is to say, to apply Paxos in your own projects, basically you have to customize and implement a set of Paxos protocols that meet your specific requirements.

Therefore, the developers of Raft had a clear objective when designing Raft: to make this protocol more understandable and select the most understandable design solution if several Raft design solutions are available. The developers provided an example. In the leader selection phase of the Raft protocol, one possible solution is to append an ID to each server and let all servers select the server with the greatest ID as the leader to quickly reach consensus (similar to the ZAB protocol). However, this solution has one additional concept—ServerID. In the meantime, a server with a smaller ID has to wait for some time before it can be selected as a new leader if the server with a higher ID fails, which can influence availability. Raft uses a very simple leader selection solution: Each server sleeps for a certain period of time and the server that wakes up first makes a proposal; if this server receives the majority of the votes, it is selected as the leader. In a general network environment, the server that makes a vote first will also receive votes from other votes first. Therefore, only one round of votes are required to select a leader. The entire leader selection process is very simple.



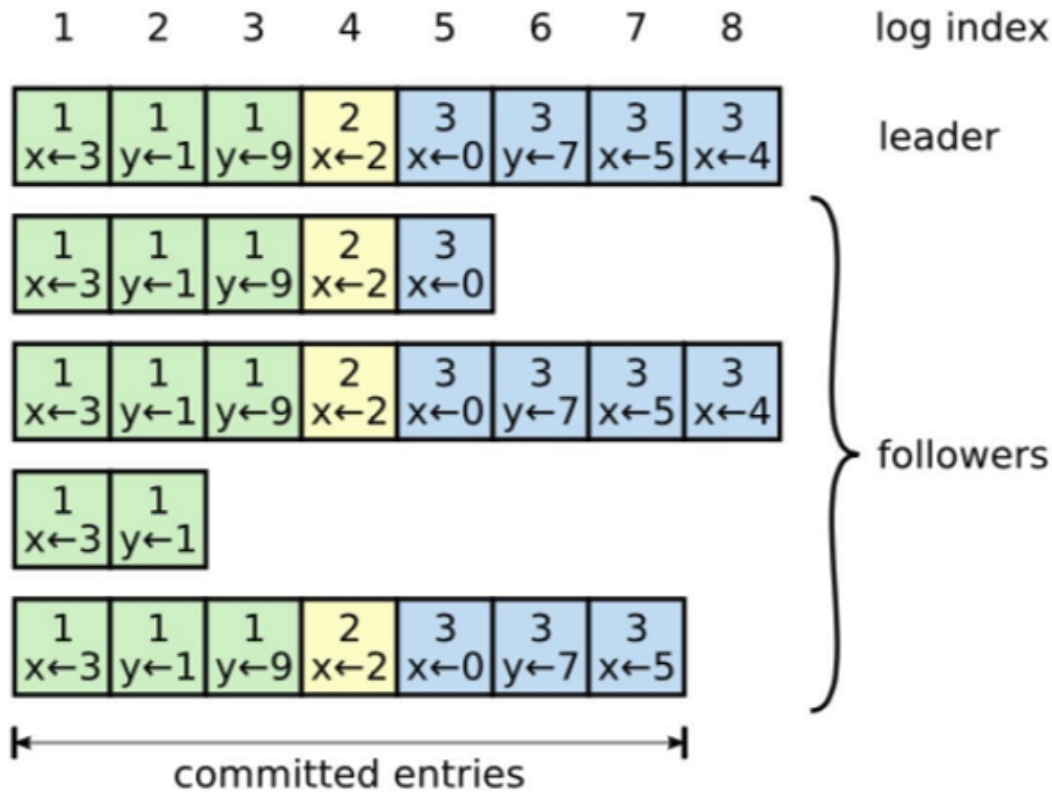
In addition to the leader selection, the overall design of the Raft protocol is also simple. A total of two RPC calls are required for interaction between the leader and followers if snapshots and changes in the number of members are not taken into consideration. One of the two calls is the RequestVote, which is only required for leader selection. That is to say, all data interactions are performed by the AppendEntries RPC.

To understand the Raft protocol algorithm, we need to look at the Term concept first. Each leader has its own term, which will be applied in each entry of the log to represent which term that entry is written in. In addition, a term is equivalent to a lease. If a leader does not send a heartbeat within a specified period of time (sending a heartbeat is also done by the AppendEntries RPC call), a follower will consider that the leader has failed and add 1 to the largest term that it has received to form a new term and start a new election. If the terms of candidates are not higher than the term of this follower, the follower will veto these candidates to ensure that the new leader selected has the highest term. If no followers receive enough votes before the timeout (this case is possible), a follower will add 1 to the current term and start a new vote request, until a new leader is selected. Raft was originally implemented in C. It is possible to set a very short timeout (usually in milliseconds). Therefore, in Raft, a failed leader can be detected in dozens of milliseconds and the fault recovery time can be very short. For databases implemented in Java using Raft (like Ratis), I estimate that it is impossible to set such a short timeout when GC time is taken into consideration.



The write operation of the leader is also a two-phase commit process. First, the leader will write the first vacant index found in its own log, and send the value of that entry to each follower through the `AppendEntries` RPC. If the leader receives "true" from more than half of the followers (including itself), in the next `AppendEntries`, the leader adds 1 to `committedIndex`, indicating that the written entry has been submitted. As shown in the following figure, the leader writes $x=4$ to the entry with `index=8` and sends it to all followers. After receiving votes from the first server (the leader itself), the third server, and the fifth server (the entry with `index=8` is not shown in the figure, but this server will definitely give a consent vote because all entries before this server are consistent with the leader), the leader obtains the majority of votes. In the next RPC, the `Committed index` will be increased by 1, representing that all the entries with `index` ≤ 8 have been submitted. The log content of the second server and the fourth server is obviously lagging behind. One reason is that after the previous RPC calls failed, the leader will retry until the logs of the followers become updated to the log of the leader. Another reason is that the two servers have been restarted and are currently in the recovery state. When the two servers receive the RPC that writes to the entry with `index=8`, a follower will also send the term and index of the last entry to these servers. That is to say, the messages with `prevLogIndex=7` and `prevLogTerm=3` will be sent to the second server. For the second server, the entry with `index=7` is empty (that is, the log is inconsistent with the leader). It will return "false" to the leader, and the leader will traverse backwards indefinitely until it finds an entry that is consistent with the log content of the second server. From that point, the log content of the leader is re-sent to the follower to complete the recovery. The Raft protocol ensures that each index in the replicated logs of all members has consistent content if their terms are consistent. If the content is not consistent, the leader will modify the content of that index to make it

consistent with the leader.



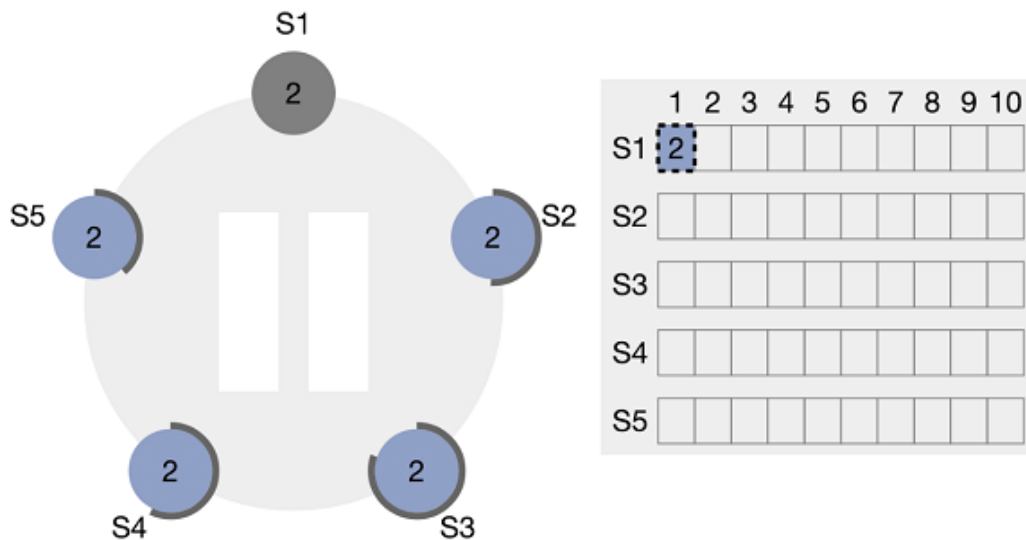
Actually my previous description has almost fully explained the leader selection, writing, and recovery in Raft. We can find some interesting aspects about Raft.

The first interesting aspect is that log entries in Raft can be modified. For example, a follower receives the Prepare request from a leader and writes the value into the index. If that leader fails, the newly elected leader may reuse this index, and the index content of this follower may be modified. This causes two problems: Logs in Raft cannot be implemented in a append-only file or file system. For ZAB and Paxos protocols, logs are only appended. This only requires a file system to have the append capability and does not need the random access and modification capabilities.

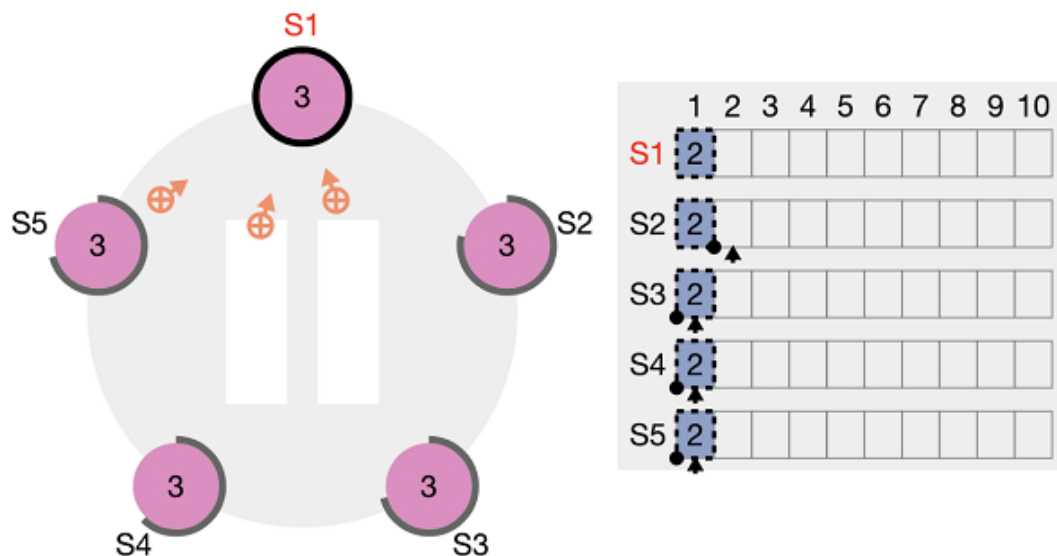
The second interesting aspect is that only one Committed index is maintained in Raft to ensure simplicity. That is, any entries that are smaller or equal to this committedIndex will be considered to have been committed. This causes the leader to fail before it receives the majority of votes (or before the leader can inform any followers that it has written its own log) during the process of writing. If this server is selected as the leader

again after restarted, this value will still be committed and made permanently valid. Because this value is included in the log of the leader, the leader will definitely ensure that the logs of all the followers are consistent with its own log. By default, this value will be committed after subsequent writes are performed and the committedIndex is increased.

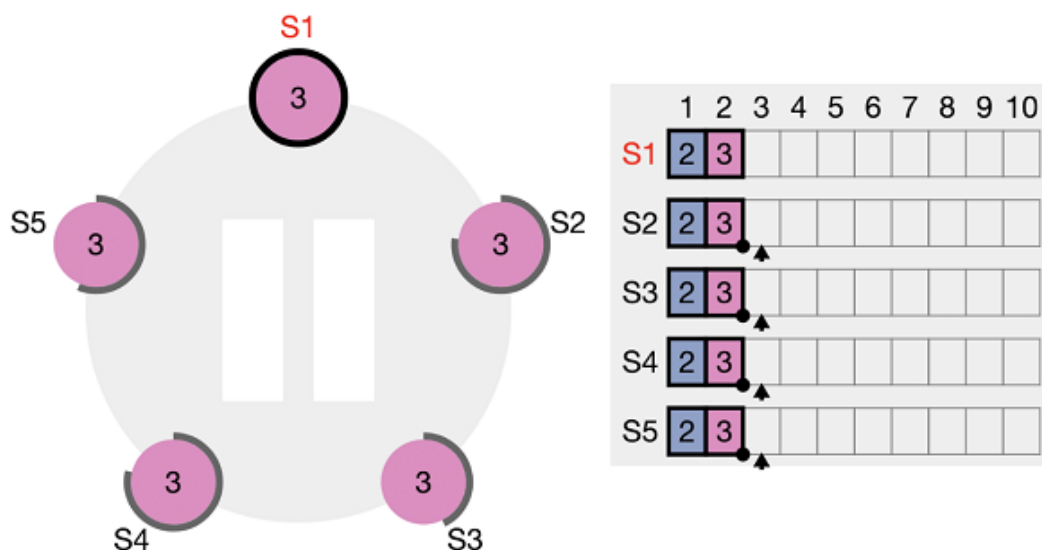
For example, consider five servers. S1 is the leader. When S1 writes the entry with index=1, it writes data into its own log first and experiences downtime before it can inform other servers of the appended entry.



After restarted, S1 may still have the chance to be elected as the leader. When S1 is selected as the leader again, it will still replicate the entry with index=1 to each server (however, the committed index will not move forward).



At this point, S1 performs another write operation. After the write is completed, the Committed index will move to the position 2. Therefore, the entry with index=1 is also considered to have been committed.



This behavior is a bit strange, because it equivalently means Raft allows a value to be committed without the consent of the majority. This behavior depends on the leader. In the preceding example, if S1 is not selected as the leader after restarted, the content of the entry with index=1 will be overwritten by the content of the new leader and the content that does not experience the voting phase will not be committed.

Although this behavior is a bit strange, it does not cause any problems,

because the leader and the follower will reach consensus. Additionally, the failure of the leader during the write process is a pending semantic to a client. The papers on Raft also say that if the "exactly once" semantic is required, a user needs to add something like UUID during the write process to allow the leader to check if the UUID has been written before the write operation. This can ensure the "exactly once" semantic to some extent.

The papers on Raft also compare Raft with the ZAB algorithm. One disadvantage of the ZAB protocol is that data exchange is required between the leader and the followers during the recovery phase. I do not really understand this. To my mind, for reselecting a leader in ZAB, the server with the largest Zxid will be selected as the leader, and other followers will complete the data from the leader—it is not the case that the leader completes its data from the followers.

Closing Words

Currently, the improved Paxos protocol has been used in many distributed products, such as Chubby, PaxosStore, Alibaba Cloud X-DB, and Ant Financial OceanBase. It is generally believed that the Raft protocol has lower performance than Paxos because it only allows committing entries in sequence. However, TiKV that uses Raft officially declares that it has made many optimizations on Raft and has significantly improved the performance of Raft. POLARDB is another Alibaba Cloud database that also uses Parallel-Raft (the improved version of Raft) to implement the parallel commit capability in Raft. I believe that more Paxos/Raft-based products will be available in the future and that more improvements will be made to Raft/Paxos.

References

1. Time, clocks, and the ordering of events in a distributed system
2. Implementing fault-tolerant services using the state machine approach- A tutorial
3. Paxos Made Simple
4. Paxos made live- An engineering perspective
5. Multi-Paxos (one PPT presentation at Stanford University)
6. Zab- High-performance broadcast for primary-backup systems

7. In search of an understandable consensus algorithm (Raft)

[Computing](#)[Logical Clock](#)[Raft](#)[ZooKeeper](#)[Paxos](#)

 0  0  0

Share on



Read previous post:

[A Basic Guide on Deploying
Apps to Container Service from
Bitbucket](#)

Read next post:

[Performance Analysis of Alibaba
Large-Scale Data Center](#)

You may also like

[Brief Introduction to Distributed
Consensus: Raft and SOFARaft](#)

block - September 14, 2021

[PolarDB: Alibaba Cloud's Relational
Database Services Architecture](#)

Alibaba Clouder - April 10, 2018

[From Distributed Consensus
Algorithms to the Blockchain
Consensus Mechanism](#)

vic - August 30, 2019

[Source Code Analysis of ClickHouse
Keeper](#)

Alibaba EMR - March 18, 2022

[Single-Step Implementation of Two-
Phase Membership Change](#)

Xiangguang - October 11, 2021

[DBMS in Review: How Alibaba Cloud
Databases Can Help Serve Your
Enterprise](#)

Alibaba Clouder - May 20, 2020

Comments

Write your comment...

Post





Alibaba Clouder

2,630 posts | 685 followers

[Follow](#)

Related Products



ECS(Elastic Compute Service)

An online computing service that offers elastic and secure virtual cloud servers to cater all your cloud hosting needs.

[Learn More](#)



OSS(Object Storage Service)

An encrypted and secure cloud storage service which stores, processes and accesses massive amounts of data from anywhere in the world

[Learn More](#)



ApsaraDB for RDS

An on-demand database hosting service for MySQL, SQL Server and PostgreSQL with automated monitoring, backup and disaster recovery capabilities

[Learn More](#)

More Posts by Alibaba ...

[See All >](#)

- [MyBatis with a More Fluent Experience](#)

- Alibaba Cloud Sustainability Report 2021
- Comparing CNI Models in Container Service for Kubernetes — Alibaba Cloud Series Part 1
- [Infographic] 5 Steps to Accelerate Your Digitalization in Asia
- Attackers Use the Vulnerability of ShowDoc to Spread Botnets
- Powerful: MyBatis and Three Streaming Query Methods
- What is the Difference between Spring Boot and Spring?
- What are the Differences and Functions of the Redo Log, Undo Log, and Binlog in MySQL?
- On the In-Depth Cluster Scheduling and Management
- Alibaba Technological Practices: Experiences in Cloud Resource Scheduling

A Free Trial That Lets You Build Big!

Start building with 50+ products and up to 12 months usage for Elastic Compute Service

Get Started for Free

[About Us](#)

[Privacy Policy](#)

[Legal](#)

[Notice List](#)

[Alibaba Group](#) [Taobao Marketplace](#) [Tmall](#) [Juhuasuan](#) [AliExpress](#) [Alibaba.com](#) [1688](#) [Alimama](#) [Alitrip](#)
[YunOS](#) [AliTelecom](#) [AutoNavi](#) [UCWeb](#) [Umeng](#) [Xiami](#) [DingTalk](#) [Alipay](#)

© 2009-2022 Copyright by Alibaba Cloud All rights reserved

